

Contents

Revisão Técnica do Relatório de Análise da Fábrica Agêntica — Opinião e Plano de Ação 1

1. Objetivo deste documento 1

2. Veredito sobre o relatório original 1

3. Minha opinião técnica priorizada 2

4. Plano de ação 3

5. O que não fazer 4

6. Conclusão 4

Revisão Técnica do Relatório de Análise da Fábrica Agêntica —
Opinião e Plano de Ação

Data: 18/08/2026 · **Revisor:** Claude Sonnet 5 (Claude Code) · **Documento revisado:** relatorios/18-08-2026-analise-fabrica-agentica.md (autor original: Solar Pro 4, via Hermes Agent) **Método:** cada citação de arquivo/linha do relatório original foi conferida contra o código-fonte real antes de qualquer conclusão ser aceita ou refutada.

1. Objetivo deste documento

O relatório original faz uma análise de arquitetura, código, segurança e UI/UX da fábrica. Antes de tratar suas conclusões como corretas, conferi as citações concretas (arquivo:linha) contra o repositório atual. Este documento registra: (a) o que se confirma, (b) o que estava impreciso, (c) um achado real que o relatório não capturou, e (d) um plano de ação priorizado.

2. Veredito sobre o relatório original

No geral: análise de boa qualidade, citações majoritariamente precisas, mas com duas imprecisões numéricas e um ponto cego relevante.

2.1 Confirmado por leitura direta do código

- 1. `compilar-para-pdf.py:97-98` — paths do Pandoc/Typst hardcoded (WinGet, usuário específico).  **Confirmado.** Linhas 97-98 fixam C:\Users\trcnologia\AppData\Local\Microsoft\WinGet\... literalmente.
- 2. `compilar-para-pdf.py:336` — "author=Heverton Eduardo Peres" fixo.  **Confirmado.** Linha 336, string literal na chamada Pandoc.
- 3. `compilar-para-pdf.py:26` — `sys.path.insert` para importar de scripts/.  **Confirmado.** Linha 26 exata.
- 4. `validar-codigo.py` — `executar_bloco` roda em `tempdir` + `env` mínimo, mas `PATH` completo é repassado.  **Confirmado, e o risco é maior do que o relatório sugere** (ver §3.1). Linhas 241-283: `env["PATH"] = os.environ.get("PATH", "")` — nenhuma restrição de rede/filesystem além do `cwd`.
- 5. `.env/.mcp.json` sem credenciais em texto puro.  **Confirmado** (o relatório original havia marcado como "não verificado"). Só existe `.env.example` (227B) no repo; `.mcp.json` sem strings de `key/token/secret`.

2.2 Impreciso no relatório original

- 1. “~30 imports de SLUGS_* via try/except (l. 41-92)” — na leitura real são 9 blocos (`dados_series`, `dados_series_perfumaria`, `dados_series_web`, `dados_series_ia`, `dados_series_stack`, `dados_livro_marketing`, `dados_series_planejamento`,

`dados_livro_deepseek, dados_series_zp`), não ~30. O número foi superestimado, mas a observação de fundo (isso deveria ser um manifesto único) está correta.

2. Nenhuma citação incorreta encontrada nas demais seções conferidas (Seção 2.4, 3.1, 3.2). As demais foram tratadas como plausíveis por leitura de contexto, sem grep linha a linha de 100% do documento — o escopo desta revisão priorizou as afirmações mais fortes (segurança e paths hardcoded).

2.3 Ponto cego do relatório original — código morto de domínio estranho

O relatório classificou os 9 imports `SLUGS_*` como “problema de legibilidade”. A checagem real mostra algo mais sério: **os 5 módulos `dados_series_perfumaria`, `dados_series_web`, `dados_livro_marketing`, `dados_livro_deepseek`, `dados_series_zp` não existem no repositório** — o `try/except ImportError` sempre cai no fallback `[]` silenciosamente.

Isso não é apenas dead code — é um **vazamento de domínio**: são nomes de outro produto inteiramente diferente (perfumaria, cursos de web fullstack, marketing digital, “segredos do DeepSeek”, “zero ao profissional”), sem relação nenhuma com a Fábrica Agêntica de Publicações (Livro/TCC/Artigo/E-book/...) descrita no `CLAUDE.md`. Duas hipóteses: (a) esse arquivo raiz `compilar-para-pdf.py` foi copiado de outro projeto e nunca higienizado, ou (b) esses módulos existiram e foram removidos sem limpar os imports. Em qualquer caso, hoje é ruído morto que:

- Confunde qualquer leitura futura do arquivo (parece que a fábrica produz 350+ livros de 5 domínios não relacionados; na verdade produz zero, pois os módulos não existem).
- Esconde silenciosamente falhas reais de import (se um desses arquivos **for** recriado com um erro de sintaxe, o `except ImportError` genérico não pega `SyntaxError`, então o comportamento muda de “lista vazia silenciosa” para “crash” sem aviso prévio no changelog).
- Não está coberto por nenhum teste que garanta a lista continua vazia (`SLUGS.extend(...)` às linhas 186-194 depende de listas vindas de módulos inexistentes).

Isto é o achado de maior prioridade prática desta revisão — é a única questão onde o código diverge do que o relatório original descreveu, e é uma remoção de baixo risco e alto ganho de clareza.

3. Minha opinião técnica priorizada

Concordo com a estrutura de 4 dimensões do relatório original (arquitetura/código/ segurança/UX), mas discordo da priorização implícita. Ordenando por impacto real:

3.1 Prioridade ALTA — sandbox de `--executar` é `cwd-isolation`, não `sandbox`

O relatório chama de “sandbox leve” corretamente, mas subestima a superfície: `env["PATH"]` é repassado por completo (linha 249 de `validar-codigo.py`), o que significa que qualquer bloco de código `python/js/bash` extraído de um capítulo/playbook e executado com `--executar` tem acesso a **qualquer binário do PATH do usuário** e pode ler/escrever fora do `tempdir` via caminho absoluto, abrir sockets, etc. O `cwd=td` só limita *onde os relativos caem*, não o que o processo pode alcançar. Para o modelo de ameaça atual (conteúdo gerado por LLMs do próprio pipeline, operador único) o risco residual é baixo — mas o relatório trata isso como “risco baixo, aceitável” sem propor mitigação de baixo custo. Existe uma: **allowlist de padrões perigosos antes de executar** (`open()`, `socket`, `subprocess`, `requests`, `os.remove`, caminhos absolutos fora do `tempdir`) rejeitando o bloco com aviso, em vez de rodá-lo cegamente. Isso não exige sandbox de verdade (container/seccomp) — é um grep de defesa em profundidade antes do `subprocess.run`.

3.2 Prioridade ALTA — remover os 5 imports mortos de domínio estranho

Ver §2.3. Ação mecânica, zero ambiguidade, remove ruído e uma fonte de confusão futura.

3.3 Prioridade MÉDIA — paths hardcoded do Pandoc/Typst e autor fixo

Concordo com o relatório: `shutil.which("pandoc")/shutil.which("typst")` com fallback para variável de ambiente (`FABRICA_PANDOC_BIN/FABRICA_TYPST_BIN`) resolve tanto a portabil-

idade Windows→outro-usuário quanto Windows→Linux/macOS. Note que gerar-relatorio-sessao.py:35-36 **já faz isso corretamente** (shutil.which("pandoc") or "pandoc") — ou seja, o padrão correto já existe no próprio repo, só não foi retroaplicado em compilar-para-pdf.py. Isso reduz o esforço da correção: copiar o padrão já validado, não inventar um novo.

O autor fixo ("Heverton Eduardo Peres") deveria vir de config_obra.json com esse valor como default apenas se o campo não existir — trivial.

3.4 Prioridade MÉDIA — checklist de segurança de deploy para a máquina de vendas

Concordo integralmente com o relatório aqui. A máquina de vendas é o único artefato da fábrica que vira uma aplicação web exposta de verdade (as demais são documentos estáticos). Um checklist mínimo documentado (rate limiting no /api/checkout, não logar payload de leads em claro, HTTPS obrigatório, autenticação no painel de leads, política de retenção) é barato de escrever e evita que o operador deploy uma máquina sem essas proteções por desconhecimento — hoje nada no fluxo /criar-maquina avisa sobre isso.

3.5 Prioridade BAIXA — schema único de config

O relatório está certo que a verdade está espalhada (config_obra.json, tipos_obra.py, metadados_livro.py, parametros_obra.py...), e que isso já causou pelo menos um bug real documentado no próprio RTK (obra_mae vs. serie/livro_mae, 2026-08-10). Mas um json-schema formal para todo o contrato de estado é esforço desproporcional ao ganho atual — a fábrica já tem um padrão mais barato e comprovado (registro declarativo central, como tipos_obra.py) que resolveu exatamente esse tipo de dispersão para o registro de tipos. Recomendo replicar esse padrão apenas para o campo que já mordeu (nomes de vínculo pai↔filho: serie/livro_mae/obra_mae) em vez de um schema geral especulativo.

3.6 Prioridade BAIXA — heurística de detectar_linguagem sem teste

Real, mas de baixo risco: o fallback é usado só quando o bloco não tem tag de linguagem (cards de playbook sem anotação), e o pior caso é classificar errado e falhar a validação (fail-safe, não fail-open). Vale 1 teste parametrizado, não uma reescrita.

3.7 Discordância pontual: “conceito de coleção/série/hub” como fricção de UX

O relatório trata isso como fricção para o operador. Na prática, o operador não navega a estrutura de pastas manualmente no fluxo normal — os comandos (/colecão, /produzir-obra-completa) abstraem isso. A fricção real só aparece quando algo dá errado e o operador precisa depurar manualmente (como os episódios registrados no RTK de artefatos gravados fora do hub). Ou seja, o problema não é o conceito ser abstrato — é a **ausência de validação automática que impeça gravação fora do hub em primeiro lugar** (o próprio relatório já sugere isso no ponto 1.3.4, “guardião de caminho”). Isso reduz duas observações do relatório (1.3.4 e 4.4.2) a uma única causa raiz, e essa causa raiz é mais barata de resolver do que “educar o operador sobre o glossário”.

4. Plano de ação

1. Imports mortos de domínio estranho — Prioridade **Alta** · Esforço baixo (mecânico) Remover os 9 blocos try/except SLUGS_* (linhas 41-92) e os SLUGS.extend(...) correspondentes (linhas 186-194) de compilar-para-pdf.py; manter só dados_series/ SLUGS_EXTRA se este existir e for usado, senão remover também.

2. Sandbox de --executar — Prioridade **Alta** · Esforço médio Adicionar allowlist de padrões perigosos (regex sobre o código antes de rodar: open(), socket, subprocess, requests, urllib, os.remove, shutil.rmtree, caminho absoluto tipo C:\ ou /etc) em executar_bloco de validar-codigo.py, recusando com mensagem clara em vez de executar.

3. Paths hardcoded Pandoc/Typst — Prioridade Média · Esforço baixo Substituir PANDOC/TYPST fixos em compilar-para-pdf.py:97-98 por shutil.which(...) or

`os.environ.get("FABRICA_PANDOC_BIN"/"FABRICA_TYPST_BIN")`, no mesmo padrão já usado em `scripts/gerar-relatorio-sessao.py`:35-36.

4. Autor fixo no template — Prioridade Média · Esforço baixo Ler `author` de `config_obra.json` (fallback para o valor atual só se ausente) em vez de string literal na linha 336.

5. Guardião de caminho do hub — Prioridade Média · Esforço médio Implementar `tipos_obra._assert_dentro_do_hub(caminho, slug)` chamado por `fatiar-obra.py`, `minerar-fontes-academicas.py` e demais scripts que gravam artefato — levanta erro se o caminho de escrita não estiver sob `dir_obra(slug)`.

6. Checklist de segurança da máquina de vendas — Prioridade Média · Esforço baixo (documentação) Adicionar seção no `CLAUDE.md` (§5, item 9) e no comando `/criar-maquina` com: `rate limiting` no `/api/checkout`, não logar dados de lead em claro, HTTPS obrigatório em produção, autenticação no painel de leads, política de retenção documentada.

7. Vínculo pai↔filho canônico — Prioridade Baixa · Esforço médio Consolidar `serie/livro_mae/obra_mae` num único campo resolvido por uma função central (já existe `resolverSerieKey` em `series_capa.py` — expandir para ser a única fonte, todos os gravadores de config passam por ela).

8. Teste de detectar linguagem — Prioridade Baixa · Esforço baixo Adicionar `tests/test_validar_codigo.py::test_detectar_linguagem` com casos `python/js/bash/ambíguo`.

9. Import opcional repetido — Prioridade Baixa · Esforço baixo Extrair `util import_opcional(nome_modulo)` usado pelos 3 imports opcionais (`metadados_livro`, `parametros_obra`, `tipos_obra`) em `compilar-para-pdf.py`.

Sequenciamento sugerido: itens 1 e 4 podem sair juntos (mesma sessão, mesmo arquivo, zero dependência). Item 3 depende do padrão do item 1 estar limpo antes (evita confundir qual `SLUGS_*` ainda é válido). Itens 2 e 5 são independentes e podem rodar em paralelo. Todo item que tocar código segue R16 (`CLAUDE.md` §1): suíte 100% antes de commit.

5. O que não fazer

- **Não** construir um json-schema geral de todo o estado da fábrica agora — não há evidência de que a dispersão atual (além do caso `serie/livro_mae`) tenha causado bugs reais o suficiente para justificar o esforço. Resolver o caso conhecido (item 7) primeiro.
- **Não** trocar o sandbox de --executar por container/VM — desproporcional ao modelo de ameaça real (conteúdo gerado pelo próprio pipeline, operador único). O allowlist de padrões (item 2) é a mitigação correta para o risco atual.
- **Não** criar uma página-índice de relatórios de sessão agora — o volume atual (dezenas de arquivos em `relatorios/` e `melhorias/`) ainda é navegável por data; revisitar só se o operador reportar dificuldade real de encontrar um relatório específico.

6. Conclusão

O relatório original (18-08-2026-analise-fabrica-agentica.md) é tecnicamente sólido e a maioria de suas citações resistiu à conferência linha a linha. As correções desta revisão são menores (uma contagem imprecisa) exceto por um ponto: o código morto de domínio estranho em `compilar-para-pdf.py` é mais grave do que “legibilidade” — é resíduo de outro produto que deveria ser removido antes de qualquer outra limpeza no arquivo. A prioridade real, na minha avaliação, está invertida em relação à ênfase do relatório original: a limpeza mecânica (itens 1, 3, 4) é barata e deveria vir primeiro; o hardening de segurança (item 2) é o único item que exige desenho, não apenas remoção/substituição.